SportsOS: An AI-Powered Sports ERP for IIT (ISM) Dhanbad

Thesis Mid-Evaluation Report

submitted in partial fulfilment of the requirements for the degree of

Integrated Master of Technology

in

Mathematics and Computing

Submitted by

Anup Kumar

Admission No.: 22JE0143

Under the guidance of

Prof. Scindhiya Laxmi

Department of Mathematics and Computing

Indian Institute of Technology (Indian School of Mines), Dhanbad

Dhanbad, Jharkhand, India

13 November 2026

ABSTRACT

Sports administration at IIT (ISM) Dhanbad depends on paper registers, spreadsheets and informal messaging, which leaves facility usage, attendance and equipment stock without real-time visibility or a reliable audit trail. This thesis designs and develops SportsOS, a role-aware sports Enterprise Resource Planning system that unifies facility booking, verified attendance, equipment inventory, tournaments, complaints, announcements and analytics for the institute's sports ecosystem.

The work is positioned in Software Engineering and Information Systems. Requirements were derived from the approved proposal and modelled as 18 stakeholder roles, 70 permissions and four scope types. The system is built as a TypeScript monorepo with a Next.js 15 web client, an Expo SDK 54 mobile client, and a NestJS 11 REST API over a 31-entity PostgreSQL schema managed with Prisma. Its main engineering contributions are: scope-aware, time-bounded role-based access control with two-person approval for privileged roles; proof-of-presence attendance combining time-based one-time passwords, GPS geofencing and device binding; database-enforced consistency for bookings and stock; and a SHA-256 hash-chained audit ledger. An optional retrieval-augmented assistant operates on permission-filtered aggregate data.

The prototype was verified on 12 November 2026 against a freshly migrated database. All 71 unit tests, 83 API end-to-end checks, 200 access-control and workflow checks covering all 18 roles, and 21 browser checks passed. The evaluation also identifies the work that remains: password reset, a shared cache, mobile device testing, live evaluation of the AI layer, validation with real institutional data, and load and penetration testing. These form the plan for the second half of the project.

Keywords: sports management information system, enterprise resource planning, role-based access control, verified attendance, geofencing, web application security, NestJS, Next.js, PostgreSQL.

TABLE OF CONTENTS

Abstract ..... **i**
List of Figures ..... **iii**
List of Tables ..... **iii**
List of Abbreviations ..... **iii**
CHAPTER 1: INTRODUCTION **1**
1.1 Background 1
1.2 Sports Ecosystem at IIT (ISM) 1
1.3 Problem Statement 2
1.4 Motivation 2
1.5 Research Objectives and Scope 2
1.6 Contributions and Report Organisation 3
CHAPTER 2: LITERATURE REVIEW **4**
2.1 Digital Transformation and Sports Information Systems 4
2.2 Enterprise Systems and Multi-Stakeholder Design 4
2.3 Access Control and Web Security 5
2.4 Comparative Analysis and Research Gap 5
CHAPTER 3: REQUIREMENTS ENGINEERING **6**
3.1 Stakeholders and Roles 6
3.2 Use Cases and Methodology 6
3.3 Functional Requirements 7
3.4 Non-Functional Requirements 8
CHAPTER 4: SYSTEM ARCHITECTURE AND DATABASE DESIGN **9**
4.1 Architecture Overview 9
4.2 Frontend, Backend and API Design 9
4.3 Database Design 10
4.4 Authentication and Access Control 11
4.5 Data Flow, Security and Deployment 12
CHAPTER 5: IMPLEMENTATION **14**
5.1 Technology Stack 14
5.2 Core Platform 14
5.3 Facilities and Booking 15
5.4 Verified Attendance 16
5.5 Inventory, Tournaments and Complaints 16
5.6 Analytics, Mobile Application and AI Features 17
5.7 Implementation Status 17
CHAPTER 6: TESTING AND EVALUATION **18**
6.1 Testing Strategy 18
6.2 Test Cases 18
6.3 Evaluation 19
CHAPTER 7: RESULTS AND FUTURE WORK **20**
7.1 Achievement Against Objectives 20
7.2 Contributions 20
7.3 Limitations 20
7.4 Future Work and Revised Roadmap 21
7.5 Conclusion 21
REFERENCES **22**

LIST OF FIGURES

Figure 3.1: Use case diagram of principal actors and use cases.....7
Figure 4.1: System architecture of SportsOS.....9
Figure 4.2: Entity relationship diagram of core entities.....11
Figure 4.3: RBAC enforcement pipeline.....12
Figure 4.4: Level-1 data flow diagram.....12
Figure 5.1: Sports Officer dashboard (demonstration data).....15
Figure 5.2: Facility booking workflow.....15
Figure 5.3: Attendance check-in validation workflow.....16
Figure 7.1: Revised development roadmap.....21

LIST OF TABLES

Table 1.1: Research objectives from the approved proposal.....2
Table 2.1: Comparison of related work with SportsOS.....5
Table 3.1: Role groups, responsibilities and default permission counts.....6
Table 3.2: Functional requirements and status.....7
Table 3.3: Non-functional requirements (ISO/IEC 25010).....8
Table 4.1: Security controls mapped to ASVS 5.0.0 areas.....13
Table 5.1: Technology stack.....14
Table 5.2: Module-wise implementation status.....17
Table 6.1: Test levels and results.....18
Table 6.2: Representative test cases.....18
Table 7.1: Achievement against objectives.....20
Table 7.2: Current limitations and planned actions.....20

LIST OF ABBREVIATIONS

Abbr.	Expansion	Abbr.	Expansion
API	Application Programming Interface	KPI	Key Performance Indicator
ASVS	Application Security Verification Standard	MIS	Management Information System
CORS	Cross-Origin Resource Sharing	ORM	Object-Relational Mapping
CSP	Content Security Policy	OTP	One-Time Password
DFD	Data Flow Diagram	OWASP	Open Worldwide Application Security Project
DTO	Data Transfer Object	QR	Quick Response (code)
ERD	Entity Relationship Diagram	RAG	Retrieval-Augmented Generation
ERP	Enterprise Resource Planning	RBAC	Role-Based Access Control
GPS	Global Positioning System	REST	Representational State Transfer
HSTS	HTTP Strict Transport Security	SPEC	Sports & Physical Education Center
IS	Information Systems	SW	Students' Welfare
IST	Indian Standard Time	TOTP	Time-based One-Time Password
JWT	JSON Web Token	2FA	Two-Factor Authentication

CHAPTER 1

INTRODUCTION

1.1 Background

Institutional sport generates a continuous stream of records, from practice attendance and ground allocations to equipment loans, fixtures and complaints. Several offices must create, approve, reconcile and report on these records. When they are kept in paper registers, spreadsheets and messaging groups, the institution cannot see its operations in real time, audit them afterwards or plan from evidence.

This thesis addresses that problem for IIT (ISM) Dhanbad through **SportsOS**, a role-aware sports Enterprise Resource Planning (ERP) system comprising a web application, a mobile application and a REST API over a single relational database. The work belongs to Software Engineering and Information Systems. Artificial-intelligence features form an optional layer above the transactional core and are reported only to the extent verified. This report presents the state of the project on 13 November 2026, three months into the nine-month plan of the approved proposal [1].

1.2 Sports Ecosystem at IIT (ISM)

Sport at the institute is administered under the Dean (Students' Welfare) through the Sports & Physical Education Center (SPEC). SPEC's remit includes intramural competitions such as the **Annual Sports Meet** and the **Inter-Hostel General Championship** [2].

The facilities listed by SPEC [3] are:

- a Lower Ground with a football ground and a floodlit eight-lane, 400 m track;
- a Sports Complex with floodlit basketball, volleyball and tennis courts;
- a Students Activity Centre with a gymnasium, four badminton courts, a yoga centre, a boxing ring and a billiards room;
- a 50 m swimming pool.

Participation is competitive, inter-hostel and recreational, and SportsOS models all three.

1.3 Problem Statement

The approved proposal identifies five operational areas that currently rely on registers, spreadsheets and messaging groups [1]: facility allocation, participation and attendance, tournaments, equipment, and administrative coordination. This fragmentation causes three problems:

- administrators lack real-time visibility of facility usage, attendance and stock;
- records are vulnerable to manipulation, such as proxy attendance;
- audit and analysis are impractical.

The institute therefore needs an integrated, role-aware web and mobile ERP that converts these operations into reliable, auditable data, while giving each stakeholder access only to the functions their role requires.

1.4 Motivation

Research on sports digitalisation has largely examined single solutions for single stakeholders and calls for multi-stakeholder work [4]. Integrated sports-facility information systems are associated with improved service quality but remain little studied [5]. Broken access control is also the leading web-application security risk [6], [7]. A campus sports ERP with eighteen roles is therefore both a practical institutional need and a demanding setting for rigorous access control.

1.5 Research Objectives and Scope

The six objectives of the approved proposal are retained unchanged (Table 1.1).

Table 1.1: *Research objectives from the approved proposal*

ID	Objective
O1	Centralise facility booking, tournaments, attendance, inventory and event management in one platform
O2	Automate scheduling, approvals, record management and reporting
O3	Provide role-based access control for students, administrators, coaches, SPEC, the Sports Council and other stakeholders
O4	Enable real-time analytics and reporting
O5	Enhance the student experience through a single portal
O6	Build a secure, scalable, future-ready system able to support AI-driven features

The scope covers authentication and scoped, time-bound role-based access control (RBAC); facilities and booking; clubs; OTP- and GPS-verified attendance; QR-based inventory; tournaments and complaints; notifications and announcements; analytics; a mobile client; and an optional AI assistant.

Payments, integration with institute single sign-on, and the use of real student data before institutional approval are out of scope.

1.6 Contributions and Report Organisation

The expected contributions are:

1. a requirements model of the sports ecosystem with 18 roles, 70 permissions and four scope types;
2. a 31-entity database design with concurrency-safe booking and inventory and a hash-chained audit ledger;
3. a working prototype across web, mobile and API tiers;
4. a verification method that tests every protected endpoint class against every role.

Chapter 2 reviews related work. Chapter 3 presents the requirements. Chapter 4 describes the architecture and database. Chapter 5 reports the implementation, Chapter 6 the testing, and Chapter 7 the results and future work.

CHAPTER 2

LITERATURE REVIEW

2.1 Digital Transformation and Sports Information Systems

Vial defines digital transformation as technology-triggered change that also reshapes an organisation's processes and structures [8]. For sports administration, this means that the approvals, escalations and reporting built around a register must be redesigned, not merely digitised.

Haffner, Oshri and Kotlarsky review information-systems research on sports digitalisation from a stakeholder perspective [4]. They find that most studies examine one solution for one stakeholder group, and they call for multi-stakeholder research. A single SportsOS booking involves a requesting student, an approving officer, gate security and leadership reading utilisation data, which is the multi-stakeholder setting they describe.

Saputra and Asmawati systematically review integrated management information systems for sports facilities, covering seven studies from 2018 to 2023 [5]. They identify inventory, reservations, cleanliness supervision and performance reporting as the integrated components, and they associate integration with improved user experience and service quality.

Shahrom and Mohamad Sukri present a swimming-facility management system for a university sport centre, with bookings, payments, role-based access control and two-factor authentication [9]. Their one-time password secures login. In SportsOS, a one-time password combined with GPS proves physical presence instead.

2.2 Enterprise Systems and Multi-Stakeholder Design

Enterprise systems integrate an organisation's information flows into a single database and process model [10], [11]. SportsOS adopts two of their properties. The first is a single shared data model, in which bookings, sessions, tournaments and complaints all reference one facility record. The second is process orientation, in which every approval or escalation is a recorded state transition.

Stakeholder theory [12] adds two requirements that the role model encodes explicitly. First, seniority does not imply operational authority: a Director views dashboards but does not

approve bookings. Second, student office-bearers act only within a limited scope, such as their own hostel.

2.3 Access Control and Web Security

The NIST RBAC model defines flat, hierarchical, constrained and symmetric RBAC [13], and it was later formalised as a proposed standard [14]. SportsOS implements core RBAC with a validated active-role selection. It deliberately omits role inheritance and adds **scoped** assignments (club, hostel, sport or facility) and **time-bounded** assignments. Separation of duty applies where it matters: proposers cannot approve their own proposals, and privileged roles require a second approver.

The security design follows OWASP ASVS 5.0.0 [7] and the OWASP Top 10 [6]. It uses bcrypt password hashing [15], JSON Web Tokens [16], input whitelisting and rate limiting, and time-based one-time passwords [17], [18] as a proof-of-presence factor.

2.4 Comparative Analysis and Research Gap

Table 2.1: *Comparison of related work with SportsOS*

Work	Setting	Attendance proof	Access control	Audit trail
Haffner et al., 2025 [4]	IS review, sports digitalisation	—	Stakeholder lens	—
Saputra & Asmawati, 2025 [5]	Review, sports facilities	Not reported	Not reported	Not reported
Shahrom & Mohamad Sukri, 2025 [9]	University pool system	No (OTP for login)	RBAC + 2FA	Not reported
Sandhu et al., 2000 [13]	RBAC reference model	—	Flat to symmetric RBAC	—
SportsOS (this work)	Institute-wide campus sports	TOTP + GPS + device binding	Scoped, time-bound RBAC	SHA-256 hash chain

The literature supplies principles but no integrated, verifiable system for an academic sports ecosystem that meets four conditions together:

1. student office-bearers hold only scoped authority;
2. attendance must be provably physical;
3. booking and stock records remain consistent under concurrent use and are auditable;
4. authorisation is tested systematically across every role.

SportsOS addresses this gap.

CHAPTER 3

REQUIREMENTS ENGINEERING

3.1 Stakeholders and Roles

Requirements were derived from the approved proposal [1]. They are written to the ISO/IEC/IEEE 29148 criteria of being necessary, unambiguous and verifiable [19], and quality attributes follow ISO/IEC 25010 [20]. Each stakeholder in the proposal's governance structure became a role, giving 17 organisational roles plus a System Administrator.

The governance hierarchy is the proposal's requirements model and is to be validated with SPEC. The system therefore uses it only to draw the organisation chart, never to grant permissions. Table 3.1 groups the roles; the permission counts are the defaults measured from the running system.

Table 3.1: *Role groups, responsibilities and default permission counts*

Group	Roles	Responsibility	Permissions
Leadership	Director, Dean (SW), President SPEC	Dashboards, escalations, approval of privileged roles and events	18–27
Administration	Sports Officer (PESO)	Facilities, bookings, inventory, staff tasks, analytics	61
Staff	Coach, Ground, Equipment, Security, Operations staff	Sessions for coached clubs, facility status, stock, gate passes, tasks	11–17
Student council	General Secretary, Sports Council, Sport Coordinator, Hostel Secretary, Media Nominee	Proposals; sport- or hostel-scoped rosters and teams; announcements	14–20
Events	Event Committee	Tournaments, fixtures, results	18
Students	Club Member, Student	Bookings, verified check-in, complaints, own records	12–13
Platform	System Administrator	Users, roles, permission matrix, imports	22

3.2 Use Cases and Methodology

Figure 3.1 shows the principal actors and use cases, all of which are implemented. The project follows an iterative, incremental process [21] framed as design science research [22]: an artefact is built to address an identified organisational problem and is then evaluated rigorously. Each increment refined the requirements, extended the schema and its migration, implemented the service with permission checks, built the user interface, and added tests before the full suite was re-run.

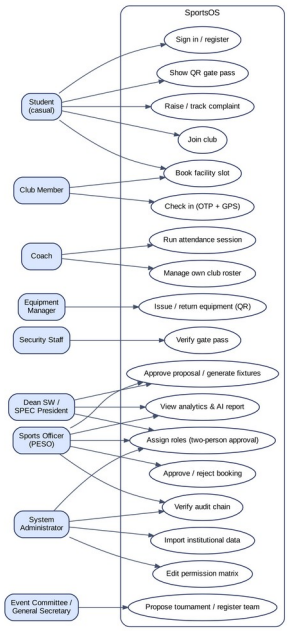


Figure 3.1: *Use case diagram of principal actors and use cases*

3.3 Functional Requirements

Table 3.2: *Functional requirements and status*

ID	Requirement	Obj.	Status
FR-01	Email and password sign-in; institute-domain self-registration yields the Student role only	O3	Implemented, tested
FR-02	Self-service password reset	O5	Not implemented
FR-03	Scoped, time-bound role assignment; privileged roles need a second approver	O3	Implemented, tested
FR-04	Run-time editable role–permission matrix with lock-out protection	O3	Implemented, tested
FR-05	Facility catalogue, availability grid and status management	O1	Implemented, tested
FR-06	Validated bookings, auto-approved or queued; QR gate pass verified at entry	O1, O2	Implemented, tested
FR-07	Coach-run sessions; check-in with rotating OTP and GPS geofence; reasoned corrections	O1	Implemented, tested
FR-08	Equipment issue and return by QR with damage and loss; stock ledger; alerts	O1, O2	Implemented, tested
FR-09	Tournament proposals, hostel-scoped registration, fixtures and results	O1	Implemented, tested
FR-10	Complaint lifecycle with escalation and attachments	O2	Implemented, tested
FR-11	Notifications, club announcements and approval-gated official announcements	O5	Implemented, tested
FR-12	KPIs, participation, utilisation, demand forecast and attendance integrity	O4	Implemented, tested
FR-13	Assistant answering questions and drafting reports from permission-filtered data	O6	Partial
FR-14	Mobile app for check-in, QR scanning, bookings and gate passes	O5	Partial
FR-15	Tamper-evident audit ledger with verification	O2, O6	Implemented, tested

3.4 Non-Functional Requirements

Table 3.3: *Non-functional requirements (ISO/IEC 25010)*

ID	Characteristic	Requirement	Status
NFR-01	Confidentiality	Password hashes, OTP secrets and device IDs are never returned	Verified
NFR-02	Authenticity	A valid token is needed on every protected route; role changes apply immediately	Verified
NFR-03	Integrity	No double-booking and no negative stock under concurrent requests	Verified (small scale)
NFR-04	Accountability	Every state change is recorded in a hash-chained ledger	Verified
NFR-05	Maintainability	Typed end to end; shared role definitions; lint-clean	Verified
NFR-06	Performance	Pages respond within 2 s for 500 concurrent users	Not tested
NFR-07	Privacy	Minimal data shown to security staff; AI never receives personal records	Verified

CHAPTER 4

SYSTEM ARCHITECTURE AND DATABASE DESIGN

4.1 Architecture Overview

SportsOS is a three-tier system (Figure 4.1). A Next.js web client and an Expo mobile client call a NestJS REST API, which persists all state in PostgreSQL through the Prisma ORM. A shared TypeScript package defines roles, permissions and data types once for both server and clients.

The API is a **modular monolith** [23] of 20 modules, each consisting of a controller, a service and validated data-transfer objects. A monolith was chosen for three reasons:

- related changes must commit atomically (a booking and its audit entry, for example);
- the load of a single institute does not justify microservices;
- one deployable unit suits a small campus IT team.

The Claude API and the Pinecone vector database are optional, and the system operates fully without them.

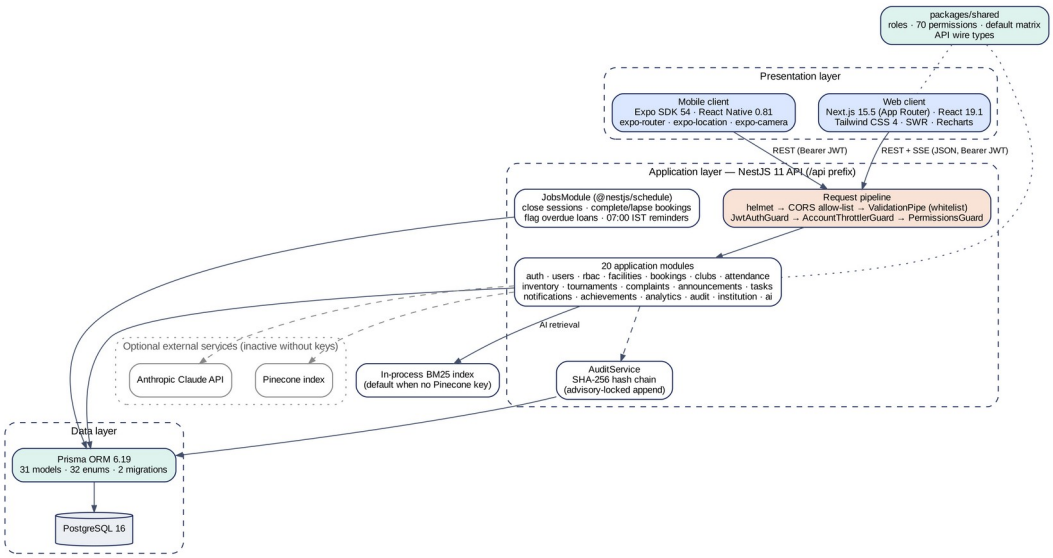


Figure 4.1: *System architecture of SportsOS*

4.2 Frontend, Backend and API Design

Web client. The web client uses the Next.js 15 App Router [24]. Its authenticated pages share a shell with permission-driven navigation, notifications and an active-role switcher,

and the production build contains 28 routes. Navigation is a usability aid only: every action is authorised again on the server.

API pipeline. Each request passes through security headers (helmet), a CORS allow-list and a global validation pipe that rejects unknown fields. Three global guards then run in order: authentication, per-account rate limiting, and permission checking [25]. Services hold the business rules and use database transactions for every multi-row change.

Endpoints. The REST API exposes 143 route handlers. State transitions are modelled as sub-resource actions, such as `POST /bookings/:id/approve`, and each maps to one audit entry.

4.3 Database Design

The schema defines **31 models and 32 enumerations** across two migrations. Figure 4.2 shows the core entities. Integrity is enforced by the database itself in four ways:

- **Unique constraints**, for example at most one attendance record per student per session;
- **PostgreSQL advisory locks** [26], which serialise bookings per facility and appends to the audit chain;
- **Conditional updates** of the form `UPDATE … WHERE availableQty >= n`, so stock can never become negative;
- **Append-only history** for role assignments, stock transactions and attendance corrections.

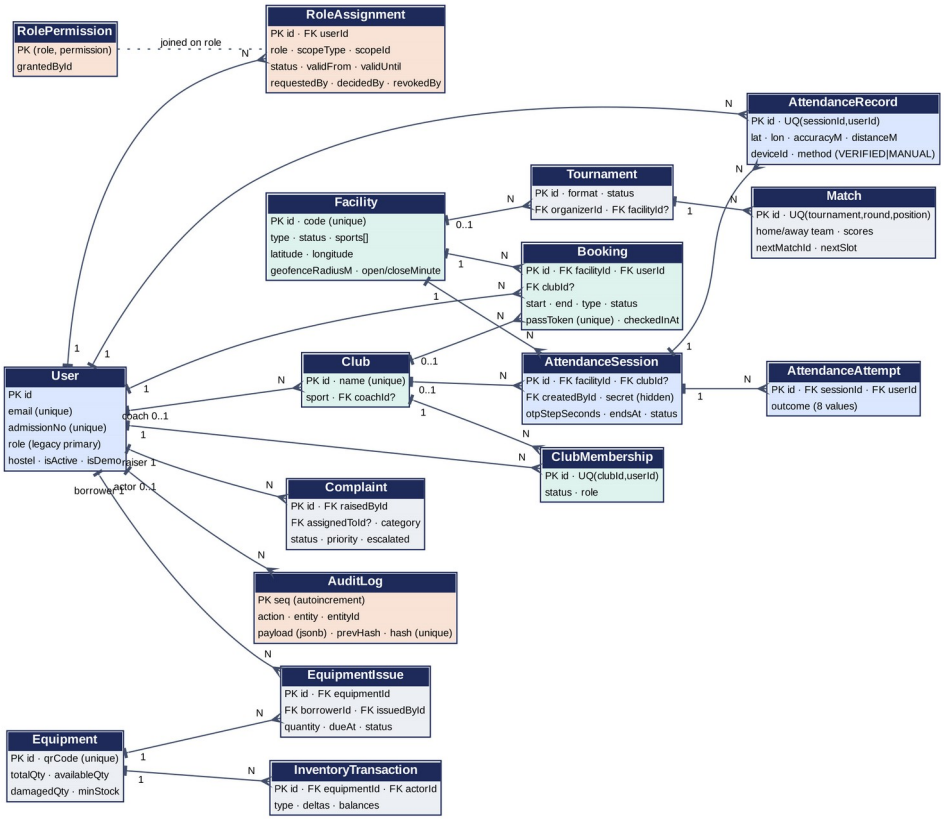


Figure 4.2: *Entity relationship diagram of core entities*

4.4 Authentication and Access Control

Authentication. Passwords are stored as bcrypt hashes. Login is rate-limited, and its response timing does not reveal whether an account exists. A successful login returns a JSON Web Token valid for 12 hours. On every request the server reloads the user's roles and permissions from the database rather than trusting the token, so revocation and deactivation take effect immediately.

Access control. Figure 4.3 shows the enforcement pipeline. An assignment links a user to one of 18 roles, together with a scope, a validity window and a status. The role–permission matrix (70 permissions and 346 default grants) is stored in the database and is editable at run time.

A pure, unit-tested function resolves each request's permissions. Route guards check the permission, and services additionally check that the target lies within the user's scope; a coach, for example, can manage only clubs they coach. Privileged roles remain pending until a second person approves them.

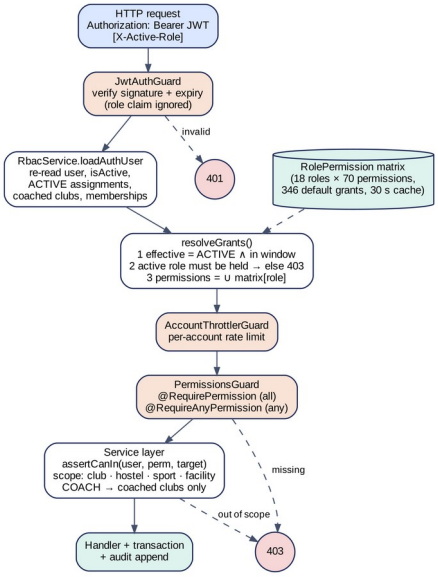


Figure 4.3: *RBAC enforcement pipeline*

4.5 Data Flow, Security and Deployment

Figure 4.4 is the level-1 data flow diagram. Booking, attendance and inventory processes append to the audit ledger, while analytics only reads operational data.

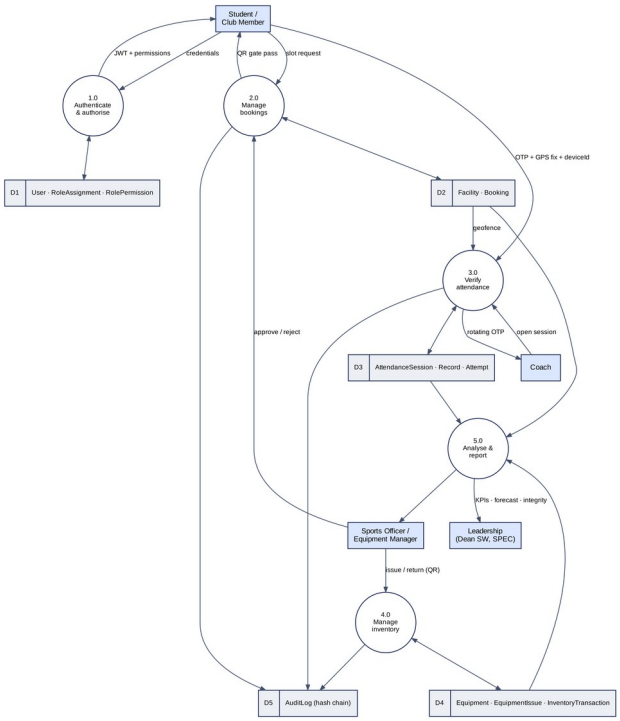


Figure 4.4: *Level-1 data flow diagram*

The implemented security controls are mapped to OWASP ASVS 5.0.0 areas in Table 4.1. The mapping is indicative and is not a formal ASVS assessment.

Table 4.1: *Security controls mapped to ASVS 5.0.0 areas*

Area	Implemented control	Open gap
Authentication	bcrypt (12 rounds); timing-equalised login; login rate limit	No password reset
Session	12-hour token; principal reloaded on every request	Web token in browser storage
Authorisation	Permission guards; object-level scope checks; spoofed roles refused	No penetration test
Validation	Whitelisted DTOs; parameterised queries; upload content checks	—
Configuration	Security headers (CSP, HSTS, nosniff); CORS allow-list	Dependency advisories
Audit	SHA-256 hash-chained audit log with verification endpoint	—

For deployment, Docker Compose defines three containers (PostgreSQL 16, the API and the web server). The API applies database migrations automatically on start-up. Cloud hosting is planned for the hardening phase.

CHAPTER 5

IMPLEMENTATION

5.1 Technology Stack

The repository is an npm-workspaces monorepo of about 28,000 lines of TypeScript, comprising the API, the web client, the shared package and the separately installed mobile app. Table 5.1 summarises the stack.

Table 5.1: *Technology stack*

Layer	Technology	Use in SportsOS
Web	Next.js 15.5, React 19.1, Tailwind CSS 4, SWR, Recharts	Role-aware pages, data fetching, charts
Mobile	Expo SDK 54, React Native 0.81	GPS check-in, QR scanning, secure token storage
API	NestJS 11 (JWT, throttler, scheduler)	Modules, global guards, scheduled jobs
Data	Prisma 6.19, PostgreSQL 16	Typed queries, migrations, transactions, advisory locks
Security	bcrypt, helmet, class-validator	Hashing, headers, input validation
AI (optional)	Anthropic SDK, Pinecone	Question answering and reports
Infrastructure	Docker Compose	Containerised local deployment

5.2 Core Platform

Dashboards. The web client provides 24 authenticated pages. A role-aware dashboard (Figure 5.1) assembles each role's work queues: approvals and live sessions for the Sports Officer, upcoming bookings for students, and the entry log for security staff.

Services. Controllers declare the permissions each route requires. Services enforce ownership, scope and state rules, and they append an audit entry inside the same transaction as each change. Four scheduled jobs close expired sessions, complete or lapse bookings, flag overdue loans and send daily reminders.

Users and roles. Login, registration and user administration are implemented, and roles change only through RBAC assignments. Administrators manage the 18 × 70 permission matrix and scoped or temporary roles from the web interface.

Demonstration data. A deterministic seed generates synthetic demonstration data, including 79 users, 9 facilities, 2,798 bookings and 1,080 check-ins. Every seeded record is flagged as demo data.

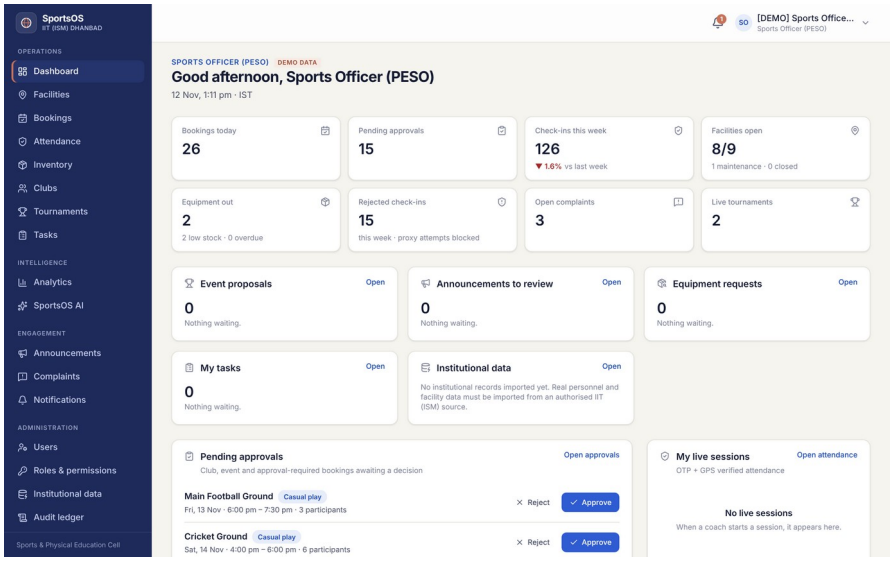


Figure 5.1: *Sports Officer dashboard (demonstration data)*

5.3 Facilities and Booking

Each facility stores opening hours, slot length, capacity, an approval flag and its GPS geofence. A booking request follows the workflow in Figure 5.2:

1. validate duration, booking horizon, opening hours and capacity;
2. lock the facility and reject any overlapping slot;
3. auto-approve eligible casual bookings, or queue the request for an approver;
4. issue a QR gate pass, which security staff scan at entry.

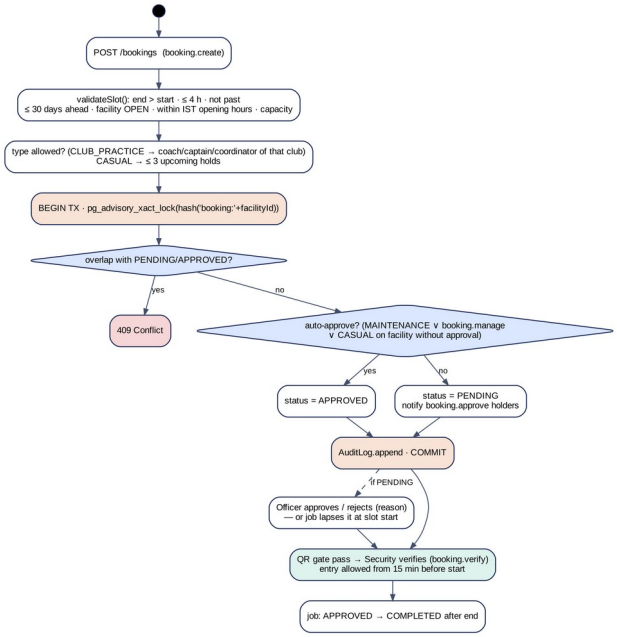


Figure 5.2: *Facility booking workflow*

5.4 Verified Attendance

A coach opens a session for a facility and club, and the server generates a secret key that is never sent to any client. The coach's screen displays a six-digit time-based one-time password (TOTP) that changes every 30 seconds [17], [18].

A student checks in by submitting the code, a GPS fix with its reported accuracy, and a device identifier. The server applies the checks in the order shown in Figure 5.3. The geofence test uses the haversine distance [27] and accepts a position within the facility radius plus min(accuracy, 25 m); fixes less accurate than 100 m are rejected. A device may check in only one student per session, and reuse alerts the coach.

Every attempt, accepted or rejected, is stored and feeds an attendance-integrity report. Manual corrections require a reason and never record fabricated coordinates.

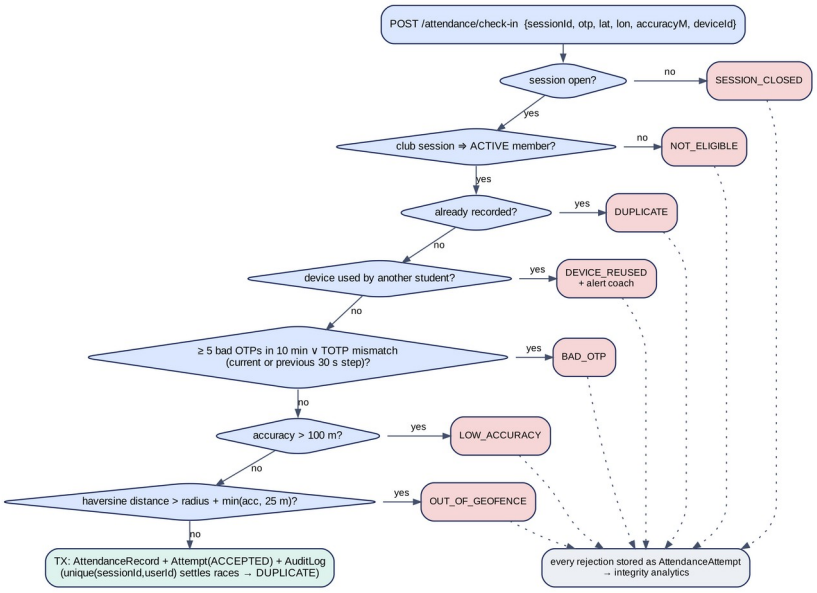


Figure 5.3: *Attendance check-in validation workflow*

5.5 Inventory, Tournaments and Complaints

Inventory. Each equipment item carries a QR code and available, damaged and total quantities. Every stock change passes through a single function that applies a conditional update and writes a ledger entry with the resulting balances. A return records good, damaged and lost units together, and low-stock and overdue alerts are raised automatically.

Tournaments. Tournament proposals require approval by someone other than the proposer, and hostel secretaries register only residents of their own hostel. Fixtures are

generated as seeded single elimination, with byes and automatic advancement, or as round robin with 3/1/0 standings.

Complaints. Complaints move through open, in-progress, resolved and closed states. Resolution requires a note, the complainant cannot resolve their own complaint, and escalation brings it to the Dean (Students' Welfare).

5.6 Analytics, Mobile Application and AI Features

Analytics. The analytics module provides KPIs, participation trends, utilisation heatmaps, equipment wear, attendance integrity and a Holt–Winters demand forecast [28], [29].

Mobile app. The Expo app provides check-in, QR scanning, bookings and gate passes across 11 screens. It passes strict type checking but has not yet been run on physical devices.

AI assistant. The assistant uses retrieval-augmented generation [30] over documents containing aggregate figures only. Documents are filtered by the user's permissions before retrieval, so restricted data never reaches the language model. Without a vector-database key, retrieval uses an in-process BM25 index [31]; without a language-model key, the assistant returns the retrieved facts and a templated report. The offline mode has been verified. The live language-model path has been tested only with mocked clients.

5.7 Implementation Status

Table 5.2: *Module-wise implementation status*

Module	Status
Authentication, users, RBAC	Implemented, tested
Facilities, booking, QR gate pass	Implemented, tested
Clubs and rosters	Implemented, tested
OTP + GPS attendance	Implemented, tested
Inventory ledger and QR	Implemented, tested
Tournaments, complaints, announcements, tasks	Implemented, tested
Analytics and audit ledger	Implemented, tested
AI assistant and reports	Partial (offline mode verified)
Mobile application	Partial (type-checked)
Password reset; shared cache (Redis)	Not implemented
Cloud deployment	Planned

CHAPTER 6

TESTING AND EVALUATION

6.1 Testing Strategy

All tests were executed on 12 November 2026 against production builds and a freshly migrated database seeded with demonstration data. The strategy has three layers:

- **unit tests** for security-critical logic: OTP generation, geofencing, hash chaining, fixtures, forecasting and permission resolution;
- **API tests** against the running server and a real database;
- **browser tests** of each role's interface.

Because broken access control is the leading risk [6], the access-control suite calls 23 protected endpoints as every one of the 18 roles. Permitted roles must not be refused, and all other roles must receive HTTP 403. Table 6.1 summarises the results.

Table 6.1: *Test levels and results*

Level	Scope	Result
Static analysis	TypeScript and ESLint across API, web and mobile	0 errors
Unit tests	8 suites	71 / 71 passed
API end-to-end	All modules	83 / 83 passed
Access control and workflows	23 endpoints × 18 roles, plus workflows	200 / 200 passed
Browser	9 role views and 12 interactive flows	21 / 21 passed
Demo accounts	Sign-in for every seeded role	19 / 19 passed

6.2 Test Cases

Table 6.2 lists representative test cases for the key scenarios.

Table 6.2: *Representative test cases*

ID	Scenario	Expected	Actual	Status
T-01	Login with valid credentials (19 accounts)	HTTP 200 + token	19 / 19 succeeded	PASS
T-02	Login with an unknown account	HTTP 401	401	PASS
T-03	API call without a token	HTTP 401	401	PASS
T-04	Student requests analytics	HTTP 403	403	PASS
T-05	Student claims the Sports Officer role in the request header	HTTP 403	403	PASS
T-06	Coach opens a session for a club they do not coach	HTTP 403	403	PASS
T-07	Six concurrent requests for one slot	One succeeds	1 × 201, 5 × 409	PASS
T-08	Check-in with the correct code from 1.1 km away	Rejected	OUT_OF_GEOFENCE	PASS

ID	Scenario	Expected	Actual	Status
T-09	Check-in with the correct code inside the geofence	Accepted	ACCEPTED (11 m)	PASS
T-10	Second check-in by the same student	Rejected	DUPLICATE	PASS
T-11	Eight concurrent issues against three units	Three succeed	3 × 201, 5 × 409	PASS
T-12	Return with one damaged unit	Stock updated	Available 13 → 12	PASS
T-13	Complaint resolved, then reopened	Status transitions	As expected	PASS
T-14	Audit chain verification	Valid	Valid (589 entries)	PASS
T-15	Forgot-password request	Reset flow	Not available	Not impl.

6.3 Evaluation

Every executed test passed. The concurrency tests (T-07 and T-11) confirm that the locking and conditional-update design prevents double-booking and stock overdraw. The access-control suite confirms that every role is confined to its permissions and scopes. Response headers include CSP, HSTS and X-Frame-Options, and requests from origins outside the allow-list are refused.

This evaluation is preliminary. It establishes functional and authorisation correctness at prototype scale. It does not yet cover performance under load, GPS behaviour on real grounds, mobile usability, the quality of AI answers, or penetration testing. A dependency audit also reported ten advisories (seven high), which are scheduled for triage.

CHAPTER 7

RESULTS AND FUTURE WORK

7.1 Achievement Against Objectives

In three months the project has progressed from proposal to a tested prototype with 31 database models, 143 API endpoints, 28 web routes, a mobile client and 375 passing automated checks. The September to November milestones are complete, and the December tournament milestone was delivered early. Table 7.1 assesses each objective.

Table 7.1: *Achievement against objectives*

Obj.	Evidence	Assessment
O1	One schema and API for all operational modules	Met
O2	Approval workflows, scheduled jobs, conflict detection	Met
O3	18 roles, 70 permissions, scoped and time-bound; 200/200 checks	Met
O4	Dashboards and forecast on demonstration data	Partly met
O5	Student web portal; mobile client not yet device-tested	Partly met
O6	Modular, containerised, AI-ready; no cloud deployment or load test yet	Partly met

7.2 Contributions

1. Scope-aware, time-bounded RBAC with two-person approval for privileged roles and no implicit seniority.
2. Proof-of-presence attendance combining TOTP, accuracy-aware geofencing, device binding and a complete attempt log.
3. Database-enforced consistency for bookings and stock, with a reconciled ledger and a hash-chained audit log.
4. A role-exhaustive verification method covering all 18 roles.

7.3 Limitations

Table 7.2: *Current limitations and planned actions*

Limitation	Planned action
No self-service password reset	Email-based reset once a mail channel is approved
Web token kept in browser storage	Move to secure httpOnly cookies
No shared cache (Redis)	Redis-backed rate limiting and caching
Mobile app not tested on devices	Android and iOS device testing
AI verified in offline mode only	Evaluation with live service keys

Limitation	Planned action
Demonstration data and placeholder geofences	Ground survey and verified institutional data
No load or penetration testing	Load tests, ASVS review and staging deployment

7.4 Future Work and Revised Roadmap

Because the core modules were completed early, the remaining months are re-planned towards validation and hardening (Figure 7.1):

- **November–January:** validation with SPEC and a ground survey;
- **December–January:** security hardening;
- **January–February:** live AI evaluation;
- **February:** mobile device testing;
- **March–April:** load and penetration testing, deployment and the final thesis.

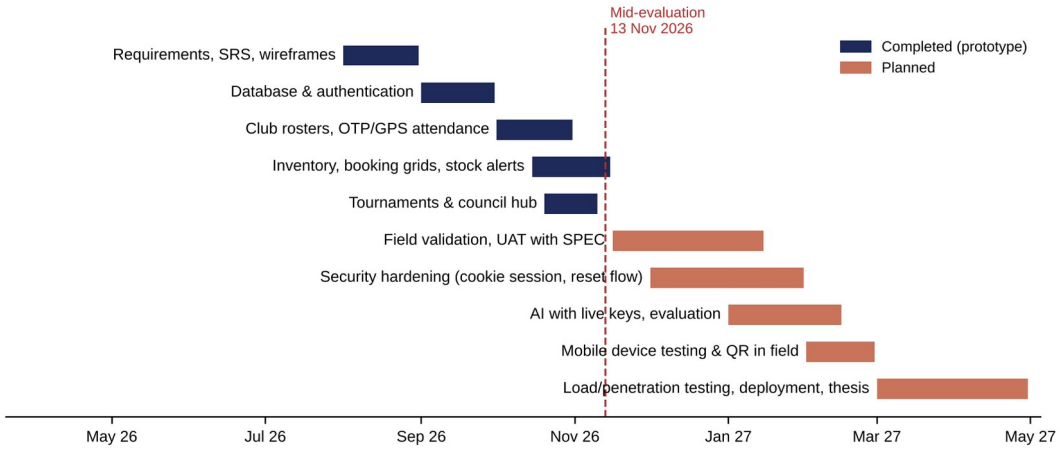


Figure 7.1: *Revised development roadmap*

7.5 Conclusion

SportsOS has progressed from proposal to a working prototype that unifies facility booking, verified attendance, inventory, tournaments, complaints and analytics for eighteen stakeholder roles. Automated tests verify its central claims:

- access is confined to each role's permissions and scopes;
- attendance requires physical presence;
- data remains consistent under concurrent use;
- every change is recorded in a verifiable audit chain.

The remaining work, comprising stakeholder validation, field testing and security hardening, is clearly scoped for the second half of the project.

REFERENCES

[1] A. Kumar, "SportsOS: An AI-Powered Sports ERP for IIT (ISM) Dhanbad," Integrated M.Tech thesis proposal, Dept. of Mathematics and Computing, IIT (ISM) Dhanbad, Aug. 2026.

[2] Sports & Physical Education Center (SPEC), IIT (ISM) Dhanbad, "Activities." [Online]. Available: https://people.iitism.ac.in/~dsw/spec/activity.html (accessed Nov. 12, 2026).

[3] Sports & Physical Education Center (SPEC), IIT (ISM) Dhanbad, "Facilities." [Online]. Available: https://people.iitism.ac.in/~dsw/spec/facilities.html (accessed Nov. 12, 2026).

[4] L. Haffner, I. Oshri, and J. Kotlarsky, "Directions for future IS research on sports digitalisation: A stakeholder perspective," *The Journal of Strategic Information Systems*, vol. 34, no. 2, Art. no. 101905, 2025, doi: 10.1016/j.jsis.2025.101905.

[5] D. E. W. Saputra and P. Asmawati, "Elevating user experience and service excellence in sports facilities via integrated management information systems: A systematic review," *Sports Science and Health*, vol. 15, no. V, pp. 111–118, 2025, doi: 10.7251/SSH25V111S.

[6] OWASP Foundation, *OWASP Top 10: 2021*. [Online]. Available: https://owasp.org/Top10/

[7] OWASP Foundation, *Application Security Verification Standard (ASVS)*, version 5.0.0, May 2025. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

[8] G. Vial, "Understanding digital transformation: A review and a research agenda," *The Journal of Strategic Information Systems*, vol. 28, no. 2, pp. 118–144, 2019.

[9] M. D. Shahrom and K. A. Mohamad Sukri, "Swimming facilities management system for Universiti Tun Hussein Onn Malaysia Sport Centre with two-factor authentication," *Applied Information Technology and Computer Science*, vol. 6, no. 2, pp. 625–643, 2025.

[10] T. H. Davenport, "Putting the enterprise into the enterprise system," *Harvard Business Review*, vol. 76, no. 4, pp. 121–131, 1998.

[11] H. Klaus, M. Rosemann, and G. G. Gable, "What is ERP?," *Information Systems Frontiers*, vol. 2, no. 2, pp. 141–162, 2000.

[12] R. E. Freeman, *Strategic Management: A Stakeholder Approach*. Boston, MA, USA: Pitman, 1984.

[13] R. Sandhu, D. Ferraiolo, and R. Kuhn, "The NIST model for role-based access control: Towards a unified standard," in *Proc. 5th ACM Workshop on Role-Based Access Control (RBAC '00)*, Berlin, Germany, 2000, pp. 47–63, doi: 10.1145/344287.344301.

[14] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, 2001.

[15] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, Monterey, CA, USA, 1999, pp. 81–91.

[16] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[17] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache, and O. Ranen, "HOTP: An HMAC-based one-time password algorithm," IETF RFC 4226, Dec. 2005.

[18] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-based one-time password algorithm," IETF RFC 6238, May 2011.

[19] *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE 29148:2018, 2018.

[20] *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*, ISO/IEC 25010:2023, 2023.

[21] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[22] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[23] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA, USA: Addison-Wesley, 2021.

[24] Vercel, Inc., *Next.js Documentation — App Router*. [Online]. Available: https://nextjs.org/docs/app

[25] NestJS, "Guards," *NestJS Documentation*. [Online]. Available: https://docs.nestjs.com/guards

[26] The PostgreSQL Global Development Group, "Explicit locking — Advisory locks," *PostgreSQL 16 Documentation*, §13.3.5. [Online]. Available: https://www.postgresql.org/docs/16/explicit-locking.html

[27] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[28] P. R. Winters, "Forecasting sales by exponentially weighted moving averages," *Management Science*, vol. 6, no. 3, pp. 324–342, 1960.

[29] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

[30] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474.

[31] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.